カーネルおよびハイパーバイザでの擬似乱数捕捉による Linux ランサムウェア被害後のファイル復号手法

北 健太朗[1,a)]　村上 洸介[1]　披田野 清良[1]

概要：ランサムウェア被害は深刻化を続けており，標的が業務システムを支える Linux システムにも広がっている．従来対策は攻撃検知とバックアップが中心であったが，検知の失敗・遅延やバックアップ自体の暗号化といった課題が残る．一方で，強力な暗号化のために，ランサムウェアは OS やハードウェアから取得した擬似乱数をもとに暗号化鍵を導出する．本稿ではこの特徴に着目し，ランサムウェアがファイル暗号化時に取得する擬似乱数を平時から記録し，被害発生後にその記録から暗号化鍵を再構成してファイルを復号する手法を提案する．まず，Qilin や DragonForce を含む実ランサムウェアの解析を通じて，事後復号の成否が擬似乱数取得経路の観測可能性と鍵導出過程の再現可能性で決まることを示す．次に，Linux システム上の擬似乱数取得経路を整理した上で，システムコールおよび CPU 命令を通じて取得される擬似乱数を，それぞれカーネル層とハイパーバイザ層で記録するシステムを設計・実装し，被害後のファイル復号が実現可能であることを示す．

キーワード：ランサムウェア，マルウェア，擬似乱数，eBPF，ハイパーバイザ

Post-incident File Decryption against Linux Ransomware by Capturing Pseudorandom Numbers at the Kernel and Hypervisor Layers

Kentaro Kita[1,a)]　Kosuke Murakami[1]　Seira Hidano[1]

Abstract: Ransomware attacks continue to grow in severity, and their targets have expanded to Linux systems. Existing countermeasures have relied on attack detection and file backups; however, detection can fail or be delayed, and the backups can also be encrypted. To achieve strong encryption, ransomware derives its encryption keys from pseudorandom numbers obtained from the OS or the hardware. Focusing on this characteristic, this paper proposes a method that records pseudorandom numbers that ransomware obtains from infected Linux systems, and reconstructs the encryption keys from those records to decrypt the files. Through the analysis of real-world ransomware such as Qilin and DragonForce, we first show that the feasibility of post-incident decryption is determined by the observability of the pseudorandom number acquisition path and the reproducibility of the key derivation process. We then organize the pseudorandom number acquisition paths on Linux systems, and design and implement a system that records the pseudorandom numbers obtained through system calls and CPU instructions at the kernel layer and at the hypervisor layer, respectively, and demonstrate the feasibility of file decryption after a ransomware incident.

Keywords: Ransomware, Malware, Pseudorandom Number, eBPF, Hypervisor

1. はじめに

暗号化型ランサムウェアは，ファイル暗号化により重要システムやデータをアクセス不能とし，復号と引き換えに身代金を要求するマルウェアである．従来の標的は Windows 端末が主であったが，近年では Akira，RansomHub，LockBit といった主要ランサムウェアグループが Linux 向け亜種を開発していることが報告されている [1]．Linux は

[1] 株式会社 KDDI 総合研究所
KDDI Research, Inc.
[a)] ke-kita@kddi.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

サーバやクラウド基盤の中核 OS であり，ランサムウェア被害は組織の業務停止に直結する．

典型的なランサムウェアでは，ランサムウェアとともに配送された攻撃者の公開鍵でファイル暗号化鍵を暗号化することで，秘密鍵を知る攻撃者のみが鍵を取り出し，ファイルを復号可能となる．Akira[2] や Rhysida[3] など特定ファミリ向けの復号ツールが公開された事例はあるが，攻撃者の秘密鍵の漏洩や実装上の欠陥を利用したものであり根本解決には至っていない．現実的な対策は侵入検知とバックアップだが，高度化する侵入手口の検知は容易でなく，バックアップを含めて暗号化される脅威が残る．

一方，ランサムウェアによるファイル暗号化アルゴリズムに着目すると，暗号用途に耐える予測困難な鍵をソフトウェア単体で生成することは困難であるため，ランサムウェアは感染システムの OS やハードウェアから取得した擬似乱数から暗号化鍵を生成する．本稿ではこの性質に着目し，平時に擬似乱数を記録する記録フェーズと，被害後にその記録から鍵材料を再構成してファイルを復号する復号フェーズからなる手法を提案する．Linux では擬似乱数の取得経路が分散するため，記録フェーズでは，read/getrandom システムコールを eBPF を用いてカーネル層でフックし，RDRAND/RDSEED 命令はハイパーバイザ（KVM）の VM-exit で捕捉する．ユーザ空間で処理が完結する vDSO getrandom はいずれの層でも観測できないため，これを無効化して通常のシステムコール経路へ集約し，3 種の取得経路すべてを覆う．また，擬似乱数だけでは鍵導出を再現できないファミリに備え，擬似乱数を取得したプロセスのメモリダンプも保存する．復号フェーズでは，記録した擬似乱数と，必要に応じてメモリダンプから抽出した補助鍵材料を候補として鍵材料を再構成し，復号の成否をファイルタイプとエントロピーにもとづくヒューリスティックで判定する．

本研究の主要な貢献は以下の 3 つである．

- **復号可能性の類型化**：被害後の復号可否が，擬似乱数取得経路の観測可能性と鍵導出過程の再現可能性の 2 条件で決まることを示す．そのうえで，Qilin や DragonForce などの実ランサムウェアの解析にもとづき Linux システムでの擬似乱数取得経路を 3 類型，ランサムウェアの鍵導出方式を 2 類型に整理し，どこに観測点を配置すれば復号可能となるかを明らかにする．

- **擬似乱数監視システムの設計と実装**：類型化した取得経路すべてに対応する擬似乱数記録機構と，鍵導出の再現に必要な補助鍵材料をメモリダンプから抽出する機構，および記録した擬似乱数から鍵材料を再構成してファイルを復号する機構からなるシステムを設計・実装する．

表 1 ランサムウェアの暗号化アルゴリズムの類型化
Table 1 Taxonomy of encryption algorithms of ransomware.

ファミリ	取得経路	鍵導出方式	暗号
Qilin	システムコール型	PRN-as-Key	ChaCha20
Babuk	システムコール型	PRN-derived Key	Sosemanuk
Prince	vDSO 型	PRN-as-Key	XChaCha20
DragonForce	CPU 命令型	PRN-derived Key	ChaCha8

- **模擬実装による実証と運用コストの評価**：実ランサムウェアの暗号化アルゴリズムを模擬したプログラムを用いて，被害後のファイル復号可能性を実証する．さらに，提案システムによるランサムウェア対策に必要な監視オーバーヘッドやログサイズなどの運用コストを定量評価する．

2. Linux ランサムウェア

2.1 擬似乱数取得経路と鍵導出方式の類型化

ランサムウェアは OS・ハードウェアから擬似乱数を取得するため，その取得経路を観測し，擬似乱数から鍵等のファイル復号に必要なデータを復元できればファイル復号が可能となる．一方，観測点を置けない経路で取得された擬似乱数は被害後の特定が困難である．すなわち，復号可能性は，取得経路を防御側が観測し取得値を記録できるかという**擬似乱数取得経路の観測可能性**と，擬似乱数から共通鍵などを導出する処理を防御側が再現できるかという**鍵導出過程の再現可能性**の 2 つによって決まる．

Linux システムにおいて OS・ハードウェアから擬似乱数を取得する経路は，3 つに分類できる．**システムコール型**は，read により`/dev/urandom` または`/dev/random` から読み出すか，getrandom を利用する．**vDSO 型**は，カーネル 6.11 以降で追加された`__vdso_getrandom` を利用し，実行がユーザ空間で完結するためカーネル層で観測できない．**CPU 命令型**は，プロセッサ内蔵の乱数生成器から値を取得する RDRAND，RDSEED 命令である．ライブラリ経由の取得も最終的にはいずれかの経路を通過する．

また, 取得した擬似乱数（PRN: Pseudorandom Number）からファイル暗号化用データを導出する方式は，擬似乱数をそのまま鍵やナンスとして利用する **PRN-as-Key** 方式と，特定のアルゴリズムにしたがって擬似乱数から鍵やナンスを導出する **PRN-derived Key** 方式に分類できる．後者では鍵生成アルゴリズムやその入力の特定も必要となる．本章で説明する Linux ランサムウェアは，**表 1** のとおり，取得経路 3 つと鍵導出方式 2 つを網羅している．

2.2 Qilin

Qilin は 2025 年に最も活発なランサムウェアの一つと報告され，Linux や VMware ESXi 向けの ELF 型亜種が確認されている [4]．今回解析した

© 2026 Information Processing Society of Japan

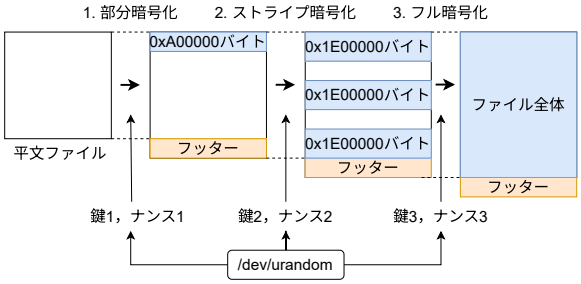


図 1　Qilin のファイル暗号化アルゴリズムの概要
Fig. 1　Overview of the file encryption algorithm of Qilin.

検体（SHA256: `a7f2a21c0cd5681eab30265432367cf4b649d2b340963a977e70a16738e955ac`）の暗号化アルゴリズムの概要を**図 1** に示す．本検体は，ファイル先頭部分の部分暗号化，一定間隔でのストライプ暗号化，ファイル全体のフル暗号化という 3 段の暗号化を各ファイルに適用する点が特徴的であり，各段では`/dev/urandom` から読み出した 32 バイト，16 バイトの擬似乱数をそれぞれ共通鍵，ナンスとして ChaCha20 暗号化するため，1 ファイルの暗号化に合計 6 つの擬似乱数を消費する．また，各段の暗号化結果末尾には，検体にハードコードされた公開鍵で共通鍵とナンスを RSA 暗号化した結果と，固定文字列`-----END CIPHERTEXT BLOCK-----`（30 バイト）を含むフッターマーカーが付与される．Qilin は PRN-as-Key 方式に該当し，システムコール経由の擬似乱数を記録できれば復号が可能となる．

2.3　Babuk

Babuk は 2021 年にビルダーコードが流出したランサムウェアであり，流出したコードは Conti，REvil，Play などのグループが ESXi 向けロッカーを開発する際に再利用された [5]．PRN-derived Key 方式の代表例であることから本研究の対象とした．攻撃者はマスター公開鍵を Babuk 検体にハードコードしておく．各ファイルの暗号化時，Babuk は`/dev/urandom` から読み出した 32 バイトの擬似乱数をファイルごとの秘密鍵とし，そこから導出した公開鍵とマスター公開鍵を用いて ECDHE（Elliptic-curve Diffie-Hellman Ephemeral）鍵共有により共有秘密を生成し，その SHA512 ハッシュの先頭 32 バイトを共通鍵として，ファイルを Sosemanuk により暗号化する．この方式では，擬似乱数がそのまま共通鍵となるのではなく，鍵導出に必要な一時秘密値として機能するため，擬似乱数を記録し，さらにメモリダンプからマスター公開鍵を抽出できれば鍵導出過程を再現できる．

2.4　Prince

Prince は，オープンソースの Go 製ランサムウェアであり，2025 年に台湾の病院などを標的とした攻撃で利用された [6]．Prince はファイル暗号化に XChaCha20 を利用し，32 バイトの共通鍵と 24 バイトのナンスに，Go の crypto/rand ライブラリを通じて取得した擬似乱数を利用する PRN-as-Key 方式である．Go 1.25.10 では，`crypto/rand.Read` は getrandom システムコールの前に`__vdso_getrandom` を実行する `runtime.vgetrandom` を試行する．vDSO getrandom はユーザ空間で処理が完結するためカーネル層では取得イベントを観測できず，vDSO 型経路の監視機構が必要となる．

2.5　DragonForce

DragonForce は 2023 年に確認され，LockBit や Conti の流出済みビルダーを再利用して多様なシステムを標的とする [7]．対象とした検体（SHA256: `188bc243cc42f8ffa4c1ed02aad5a76c9000e3d58104f45fe71af66536a274da`）では，C++標準ライブラリの `std::random_device` を通じて得た 4 バイトのシードでメルセンヌ・ツイスター擬似乱数生成器 `std::mt19937_64` を初期化し，生成した 32 バイトの共通鍵と 8 バイトのナンスを使ってファイルを ChaCha8 暗号化する．本検体の実行環境では，libstdc++の`__x86_rdseed_rdrand` を介してシードが CPU 命令から取得される．すなわち，CPU 命令型経路の擬似乱数を記録できれば `std::mt19937_64` のシードを特定でき，暗号化に利用された鍵とナンスを復元できる．

3.　提案手法

2 章で示したとおり，被害後のファイル復号可能性は，擬似乱数取得経路の観測可能性と鍵導出過程の再現可能性によって決定される．そこで提案手法は，ランサムウェアが暗号化処理のために取得する擬似乱数を平時から記録し，被害後にその記録を用いて鍵材料を再構成することで，攻撃者の秘密鍵漏洩やランサムウェア実装の脆弱性の有無に依存せずファイルを復号可能とする．

3.1　システム全体像

提案手法の全体像を**図 2** に示す．記録フェーズでは，システムコール型・vDSO 型・CPU 命令型に対応する監視機構を用意し，ユーザプロセスが利用した擬似乱数をログとして保存する．また，Babuk のように，擬似乱数監視だけでは鍵導出過程の再現可能性が満たされないファミリに備え，擬似乱数を取得したプロセスのメモリダンプを保存し，マスター公開鍵などの補助鍵材料をメモリから取得可能とする機構を加える．復号フェーズでは，記録した擬似乱数ログなどを入力としてファミリごとの鍵導出過程を再現し，暗号化ファイルの復号を試みる．

© 2026 Information Processing Society of Japan

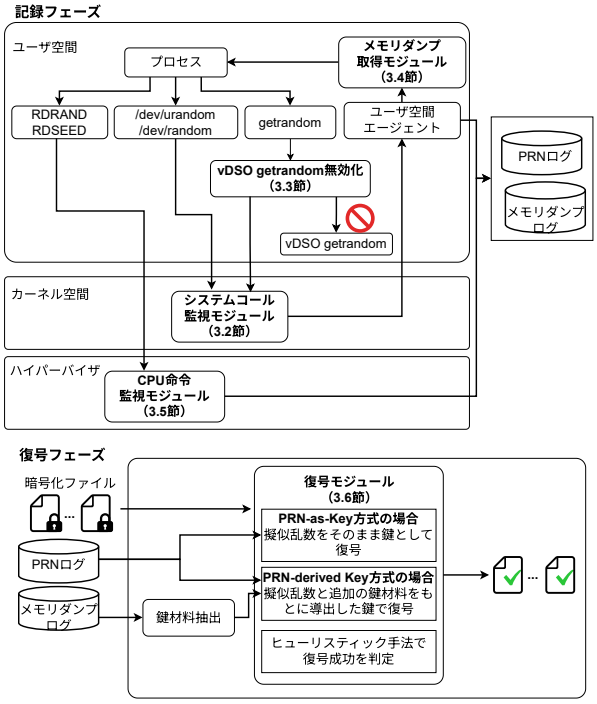


図 2　提案手法の全体像
Fig. 2　Overview of the proposed method.

3.2　システムコール監視モジュール

本モジュールは，read および getrandom のエントリーポイント（`sys_enter_read`，`sys_enter_getrandom`）とエグジットポイント（`sys_exit_read`，`sys_exit_getrandom`）を eBPF で監視することで，プロセスが OS から取得する擬似乱数を記録する．

getrandom は取得専用であるため原則すべての呼び出しを記録する一方，read は通常のファイル I/O でも呼ばれるため，エントリーポイントでファイルディスクリプタテーブルから対象ファイル名を解決し，`urandom` または `random` の場合のみ記録する．擬似乱数はエグジットポイントではじめてユーザ空間バッファに書き込まれるため，エントリーイベントで取得したバッファアドレスを eBPF マップへ一時保存し，エグジットイベントで対応するエントリを検索して `bpf_probe_read_user` でバッファ内容を読み出す．イベントには擬似乱数本体に加え，時刻やプロセス情報を含め，被害後の絞り込みに用いる．

3.3　vDSO getrandom への対応

`__vdso_getrandom` の実行はユーザ空間で完結するため，3.2 節のシステムコール監視モジュールのトレースポイントが発火せず擬似乱数を記録できない．そこで，提案手法では監視対象システムにおいて vDSO getrandom を無効化し，擬似乱数取得を通常のシステムコール経路へ戻すことで，Prince のようにライブラリがこの経路を優先的に利用する実装であっても観測可能とする．この方式では呼び出しごとのレイテンシが増加するが，OS 由来の取得は暗号ライブラリの初期化時や再シード時に限られることが多いため性能影響は限定的である．4.2 節で取得経路のレイテンシを比較する．

3.4　メモリダンプ取得モジュール

PRN-derived Key 方式のランサムウェアでは，擬似乱数監視だけでは鍵導出過程の再現が困難な場合がある．例えば，Babuk のファイルごとの秘密鍵は擬似乱数であるため観測可能である一方，マスター公開鍵は検体に埋め込まれメモリ上に展開されるため，記録した擬似乱数だけでは鍵導出過程を再現できない．そこで，擬似乱数を取得したプロセスのメモリダンプを保存し，被害後に補助鍵材料を抽出可能とする．ダンプは eBPF が擬似乱数取得イベントを検知した時点で実行するが，プロセス停止によるオーバーヘッドを抑えるため同一プロセスでは初回の 1 回のみに限定する．取得時は，不整合を防ぐため ptrace で全スレッドを停止し，`/proc/<pid>/maps` から鍵材料を含む可能性が高い区画，すなわちシステムライブラリ由来でない静的変数・定数領域と書き込み可能な匿名マッピングのみを読み取る．

被害後の解析では，保存したメモリダンプからマスター公開鍵の候補を抽出する．Babuk のマスターとなる Curve25519 公開鍵は 32 バイトのランダムに近いバイト列でありエントロピーが高い．そこで各ダンプファイルを 8 バイト境界で走査して 32 バイトのウィンドウごとにエントロピーを計算し，しきい値 T 以上の区間を候補として保存したうえで，記録済みの擬似乱数と組み合わせて復号を試行し正しい鍵材料を特定する．4.3 節での評価のとおり，T を高くすると候補数が減るため特定は高速になるが，正しい公開鍵が候補から除外される可能性が高まる．

3.5　CPU 命令監視モジュール

RDRAND/RDSEED はユーザ空間で完結しシステムコールを経由しないため，システムコールフックでは擬似乱数を観測できない．そこで，監視対象の Linux システムを KVM のゲスト VM として実行し，Intel VMX が備える VM-exit 機構を用いてこれらの命令の実行をハイパーバイザ層で観測する．具体的には，ゲスト VM ごとの実行を制御する VMCS（Virtual Machine Control Structure）の実行制御フィールドで RDRAND/RDSEED exiting を有効化して命令の実行ごとに VM-exit を発生させ，無効命令例外を注入する既定のハンドラを独自のハンドラ関数に差し替える．このハンドラは，ゲストの RIP が指す命令のバイト列をパースして命令種別・オペランド幅・宛先レジスタを識別したうえで，Linux カーネルの `get_random_bytes` で生成した擬似乱数をゲストレジスタへ書き戻し，命令成功を示す CF などのフラグを更新して，RIP を命令長分進

© 2026 Information Processing Society of Japan

めて実行を再開する．ハンドラは低レイテンシで実行される必要があるため，擬似乱数はカーネル内のリングバッファへ追加するにとどめ，ホスト側のロガーが debugfs 経由で読み出して保存する．

3.6 復号モジュール

擬似乱数ログには正規プロセスの取得値も混在し，どの擬似乱数がどのファイルに使われたかを示す情報は存在しない．そのため復号モジュールは，各暗号化ファイルに対しログ中の擬似乱数を候補として順に当てはめ，対象ファミリの鍵導出手順に従って生成した鍵で復号を試行する．PRN-as-Key 方式では，記録された擬似乱数を鍵候補・ナンス候補として直接用いる．例えば，Qilin は 3 段の ChaCha20 暗号化で各段に独立した鍵とナンスを利用するため，ログを 32 バイトの鍵候補リストと 16 バイトのナンス候補リストに分離し，各段について組み合わせを試行する．一方，PRN-derived Key 方式では，擬似乱数が鍵導出・鍵交換アルゴリズムやユーザ空間 PRNG の入力として利用される．Babuk では，各擬似乱数候補を Curve25519 の秘密鍵として扱い，マスター公開鍵候補との共有秘密の SHA512 ハッシュ前半 32 バイトを共通鍵として Sosemanuk 暗号文を復号する．DragonForce では，記録した RDRAND/RDSEED 返却値をシード候補として `std::mt19937_64` を初期化し，その出力から生成した鍵とナンスで ChaCha8 暗号文を復号する．

各試行の成否を決定論的に判定することはできないため，復号成功の判定には次の 3 つのヒューリスティックを順に適用する．第一に，2.2 節で説明した Qilin のように，復号後データ内に固定フッターマーカーが出現する場合，そのマーカーの有無を用いる．第二に，復号結果の MIME タイプが対象ファイル種別の期待型に一致するかを判定する．Babuk をはじめとするランサムウェアは，元の拡張子を残した別名（`test.pdf`→`test.pdf.babyk`）へ変換するため，元ファイルのファイルタイプを特定できる．第三に，上記が利用できない場合，全候補の中で復号後データのエントロピーが最小のものを復号成功と判定する．誤った鍵による結果はほぼ一様ランダムとなる一方，正しい平文は構造をもちエントロピーが有意に低くなるからである．復号成功と判定した場合は，利用した擬似乱数を使用済みとして以降の探索対象から除外し，探索空間を徐々に縮小する．

4. 評価

実ランサムウェアの暗号化アルゴリズムを模擬したプログラムと，対応する復号プログラムを実装し，監視のオーバーヘッドと事後復号の実現性を検証した．評価環境は Ubuntu 24.04 LTS（Linux 6.17.0-35），Intel Core i7-14700，メモリ 32 GB であり，CPU 命令監視の評価では改変した KVM 上で Ubuntu 24.04 cloud image をゲストとして実行した．

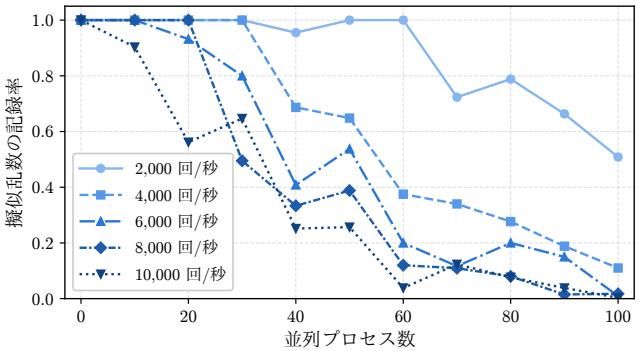


図 3 並列プロセス数・レートに対する擬似乱数の記録率
Fig. 3 PRN capture rate vs. concurrent processes and rates.

4.1 擬似乱数記録の精度と網羅性

今回の実装では擬似乱数バッファを 25 KB/CPU に設定しているため，高レートの読み出しでバッファが溢れると記録されない擬似乱数が生じ，一部ファイルの復号が困難となる．そこで，指定レートで `/dev/urandom` を読み出すプログラムをランサムウェア模擬プログラムと並列実行し，欠落頻度を測定した（**図 3**）．2,000 回/秒では並列プロセス数 60 までおおむね記録率 1.0 を維持したが，70 以上では 0.72 以下へ低下した．レートが高いほど低下が始まるプロセス数は小さくなり，10,000 回/秒では 20 プロセスで 0.56 まで低下した．なお本測定はファイル数 200・各条件 1 試行の結果である．

CPU 命令監視モジュールについては，ゲスト VM が RDRAND/RDSEED を 100 万回実行した際の記録の網羅性を評価した．RDRAND 単独，RDSEED 単独，ビット幅混在，4 スレッド並列の 4 シナリオすべてで，一定レートでの 100 万回の実行に対する取りこぼしは 0 件であり，値の一致率は 1.0 であった．一方，各スレッドがレート無制限で実行した場合，網羅率 1.0 を保てるスループットの上限は毎秒約 1.79×10^6 件であり，4 スレッドでは 0.956，8 スレッドでは 0.527 へ低下した．これは単一ロガー・単一バッファという擬似乱数読み取り側の実装に起因し，改善の余地があるが，実アプリケーションの取得頻度はこの上限よりはるかに低いことが見込まれる．

4.2 監視オーバーヘッド

提案手法は eBPF フックの追加，vDSO getrandom の無効化，命令ごとの VM-exit により性能オーバーヘッドが生じうる．そこで，システムコールは 10,000 回の連続実行から 1 回あたり，CPU 命令は 100 万命令の実行から 1 命令あたりの実行時間を算出した（**表 2**）．

read，getrandom の監視による増加は，それぞれ約 16.95%，約 14.93%であった．vDSO getrandom の無効化ではユーザ空間とカーネル空間の往復が復活するためレイ

© 2026 Information Processing Society of Japan

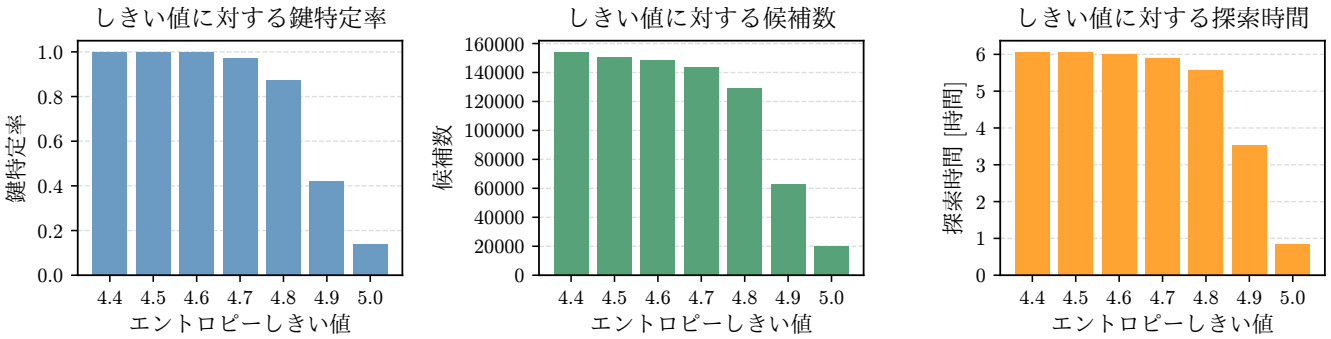


図 4　エントロピーしきい値 T に対するマスター公開鍵の特定率・候補数・探索時間

Fig. 4　Key found rate, candidate count, and search time vs. the entropy threshold T.

表 2　監視によるオーバーヘッド

Table 2　Overhead induced by the proposed monitoring.

対象	監視なし	監視あり	実行時間比
read（syscall）	639.1 ns	747.4 ns	1.17 倍
getrandom（syscall）	460.2 ns	528.9 ns	1.15 倍
getrandom（vDSO → syscall）	46.7 ns	176.7 ns	3.78 倍
RDRAND / RDSEED	310 ns	986 ns	3.18 倍

テンシが約 3.8 倍となる．なお，表 2 の vDSO → syscall の行は，取得経路そのものの変化によるレイテンシを比較した値であり，eBPF 監視によるオーバーヘッドを含まない．RDRAND/RDSEED 命令については，監視時の 1 命令あたり実行時間は約 986 ns であり，ネイティブ KVM（約 310 ns）に対する実行時間比は約 3.2 倍であった．その内訳はハンドラ内処理が約 363 ns，VM-exit/entry の往復が約 313 ns であり，後者は命令単位で記録する提案手法に内在する構造的コストである一方，前者には命令デコード結果のキャッシュなどの最適化余地がある．

また，擬似乱数ログのストレージサイズを 600 秒間測定したところ，システムコール経由では SSH サーバへ 10 秒に 1 回接続するシナリオで約 409.9 MB/日，CPU 命令経由では無負荷のゲスト VM で約 0.41 MB/日であり，前者は正規サービスの擬似乱数利用に強く依存するため，容量設計と保存期間の方針は 6 章で述べる．

4.3　復号成功率

マジックバイトをもつファイルの復号成功は MIME タイプで高精度に判定でき，例えば PDF ファイルで誤判定が生じるのは`%PDF-`が誤った復号結果の先頭に現れる確率 $1/256^5$ にとどまる．一方，MIME タイプで判定できないバイトデータなどでは，復号後データのエントロピーが最小の候補を正解と判定するため判定がより難しく，元ファイルのエントロピー H_{file} が，誤った擬似乱数による復号結果の期待エントロピー $H_{\text{wrong},n}$（サイズ n が大きいほど 8.0 に近づく）を下回る場合にのみ正しく機能する．そこで，内容分類の異なる 4 ファイルについて，誤り復号出力のエントロピー分布をモンテカルロ法で推定したうえで，

表 3　エントロピー判定の評価対象ファイルと復号成功率

Table 3　Target files and success rate of the entropy-based decision.

ファイル	サイズ	H_{file}	M=10	M=50	M=100
nvoptix.bin	129 MB	7.4800	1.0	1.0	1.0
intel-ucode	1.5 MB	7.999809	1.0	1.0	1.0
GPG 暗号文	512 KB	7.999585	0.806	0.356	0.120
AES 暗号文	512 KB	7.999684	0.0	0.0	0.0

ログにダミーの擬似乱数を $M \in \{10, 50, 100\}$ 件追記して Babuk に対する復号を 1,000 回試行した（**表 3**）．

非暗号化バイナリの nvoptix.bin と高エントロピーだがサイズの大きい intel-ucode は，H_{file} が誤り分布平均を大きく下回り全条件で成功確率 1.0 となった．一方，一様乱数を暗号化した 512 KB の暗号文では，AES 暗号文が誤り分布平均を上回るため系統的に失敗し，GPG 暗号文もダミー擬似乱数の増加に伴い成功率が低下した．すなわちエントロピー判定は非暗号化バイナリや 1 MB 以上の高エントロピーファイルに有効で，限界は全体が一様な暗号文からなる小さなファイルに現れる．

また，Babuk のようにメモリダンプからのマスター公開鍵抽出が必要なランサムウェアでは，しきい値 T が復号成否を左右する（**図 4**）．$T = 4.4, 4.5, 4.6$ の場合の鍵特定率はそれぞれ 1.0，0.999，0.998 であったが，Babuk の Curve25519 公開鍵のエントロピー分布が 4.88 付近に集中するため $T = 4.9$ では 0.422 へ急落した．探索時間は $T = 4.4$ で 6.06 時間，$T = 4.9$ で 3.54 時間であり候補数と強い正の相関が見られたが，鍵特定率を優先する場合は $T = 4.4$ または $T = 4.5$ の設定により 0.999 以上にできる．

4.4　復号にかかる時間

総当たり探索では組み合わせ数の爆発により現実的な時間で復号が完了しないことが懸念されるため，復号対象のファイル数を変化させて全ファイルの復号時間を測定した（**図 5**，**図 6**）．本評価ではログにランサムウェアが利用した擬似乱数のみが含まれるため，候補数はファイル数に比例する．Qilin は各ファイルの暗号化に 6 つの擬似乱数を

© 2026 Information Processing Society of Japan

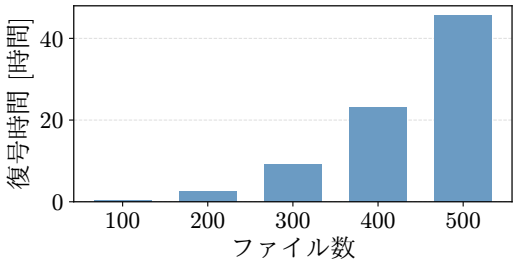


図 5 Qilin のファイル数に対する復号時間
Fig. 5 Decryption time vs. the number of files (Qilin).

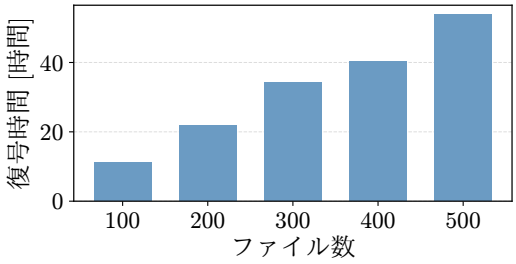


図 6 Babuk のファイル数に対する復号時間（マスター公開鍵探索にかかる時間を含む）
Fig. 6 Decryption time vs. the number of files, including the master public key search (Babuk).

利用するため，ログに記録された擬似乱数の総数を n とすると，1 ファイルあたり鍵とナンスの組の探索に $O(n^2)$，ファイル数を F として全ファイルでは $O(F \cdot n^2)$ の計算時間が必要となる．実測では，ファイル数が 100，200，300，400，500 の場合の復号時間はそれぞれ約 0.36，2.68，9.30，23.02，45.68 時間となり，ファイル数 5 倍に対して約 127 倍（$\simeq 5^3$）と急激に増大したが，いずれの場合でも全ファイルの復号に成功した．一方 Babuk の復号は 2 段階で構成され，マスター公開鍵候補の個数を m とすると公開鍵の特定に $O(m \times n)$ を要する．その後のファイル復号は，使用済みの候補を除外しながらログの記録順に探索するため，記録順が暗号化順と一致する本評価条件では $O(n)$ で完了し，ファイル数 100〜500 に対して約 1.07〜6.34 秒と軽量であった．一方，図 6 に示す公開鍵探索を含めた総所要時間は 11〜54 時間に達しており，被害後復旧の律速はマスター公開鍵の探索であることが分かる．以上はいずれも各条件 1 試行の測定値であり，かつログにランサムウェアが利用した擬似乱数のみが含まれる理想条件での下限であるため，実サービスのノイズを含む評価は今後の課題である．

5. 関連研究

擬似乱数に着目したランサムウェア対策：PayBreak[8] は，擬似乱数生成や暗号化関連の Windows API をフックして鍵情報を記録し，暗号化後のファイル復号を可能にする．鍵材料を実行時に取得できれば復旧しうることを示した点で本研究と最も近いが，Windows の暗号 API フックを前提とし Linux には利用できない．Kim ら [9] は，Windows OS 内の擬似乱数生成器を防御側が管理する決定論的なものに置き換えて取得値を後から再現し，Genç ら [10] は，無許可プロセスによる擬似乱数生成用 API 呼び出しを制御することで暗号化処理を妨げる．Elastic Security Labs[11] は，カナリアファイルへの書き込みを契機に取得したメモリスナップショットから暗号鍵の回収を試みるが，本研究は平時から記録するため検知前の鍵材料も利用できる．

Linux ランサムウェアやファイル操作に着目した検知・復旧：Brodzik ら [12] は，eBPF で収集したシステムコール情報をカーネル内の機械学習モデルに入力してランサムウェアを検知しており，本研究は eBPF を検知ではなく擬似乱数の記録機構として利用する点が異なる．GuardFS[13] はオーバーレイファイルシステムでファイルアクセスを制御し，ShieldFS[14] はファイルシステムの挙動をモデル化して検知と元ファイルの復旧を行う．

実装脆弱性を利用した復号研究：Rhysida[3] や Magniber v2[15] に対する復号研究は擬似乱数生成器の実装上の欠陥を利用して感染時点の内部状態を再生成し，Hive に対する復号研究 [16] は独自暗号アルゴリズムの脆弱性からマスター鍵の大部分を回復できると報告している．Akira の亜種でも，暗号化時刻から鍵候補を絞り込み GPU 探索で復号を試みる事例がある [2]．これらは身代金を支払わずに復号する点で本研究と目的を共有するが，成立条件が特定ファミリの実装上の欠陥に依存しており，複数ファミリに適用できない．

6. 考察

6.1 適用可能範囲

提案手法が有効に機能するには，（1）擬似乱数記録システムがランサムウェア感染前に導入されていること，（2）復号モジュールを構成するためにランサムウェアの暗号化アルゴリズムが既知であること，（3）ランサムウェアが観測可能な経路から擬似乱数を取得すること，の 3 つの前提が必要である．これらが満たされれば，暗号化処理に脆弱性がなくとも復号可能であり，個別ファミリ向けの既存復号ツールより一般化可能性が広い．ただし本稿で復号を実証したのは Qilin と Babuk の模擬実装であり，Prince と DragonForce については擬似乱数取得経路が監視可能であることの確認にとどまるため，復号の実証は今後の課題である．一方で，鍵材料を C2 サーバからダウンロードするファミリでは前提（3）が満たされない．また，攻撃者がダミーの擬似乱数を大量に生成して復号時間を増大させることも考えられるが，このような大量取得は異常なシステムコール頻度として検知できる．

6.2 実運用における考慮事項

4 章に示したとおり，擬似乱数記録のオーバーヘッドと

© 2026 Information Processing Society of Japan

ストレージコストは通常のワークロードでは許容範囲に収まるが，暗号処理を多用する環境ではログ量が増加するため，ローテーション・圧縮・リモート保管を組み合わせ，検知までのリードタイムをカバーする期間だけ保存する運用が現実的である．復号コストについては，ファイル I/O ログなどで感染時刻とプロセスを絞り込んで候補数を抑えることが有効である．また，擬似乱数ログには正規プロセスが取得した値も含まれうるため，通常ログより高い機密性をもつデータとして暗号化保存し，アクセス権と保持期間を制限する必要がある．さらに，本手法はハイパーバイザとログ保存先を信頼境界とし，ゲスト内の攻撃者がロガーやローカルログを停止・改変しうることを前提とするため，記録は append-only ログとして監視対象外の領域へ即時転送する必要がある．また擬似乱数ログとメモリダンプは正規サービスの鍵や認証情報を集約する鍵エスクローとなるため，ダンプ対象プロセスの限定と保持期間の最小化が不可欠である．

6.3 研究倫理

本研究は攻撃者に悪用され得る情報を含むため，公開する情報は防御側の実装に必要な範囲に限定し，提案手法の具体的な回避方法や解析に用いた公開コードの URL は記載しない．また評価では，安全性に配慮し，実検体を実行せず，実ランサムウェアと同一の暗号化アルゴリズムを実装しつつ，指定したディレクトリ配下の特定の拡張子をもつファイルのみを暗号化するよう機能を限定した無害化プログラムを用いた．

7. おわりに

本稿では，ランサムウェアが鍵生成に用いる擬似乱数に着目し，Linux ランサムウェアに暗号化されたファイルを身代金の支払いなしに復旧する手法を提案・実装した．被害後の復旧可否を擬似乱数取得経路の観測可能性と鍵導出過程の再現可能性に帰着させて類型化したうえで，read/getrandom を eBPF で，RDRAND/RDSEED をハイパーバイザ層で記録し，vDSO 経由の取得は無効化により前者へ集約する監視システムを実装し，Qilin・Babuk の暗号化アルゴリズムを模擬した実装により事後復号を実証した．本手法は既存の検知・防御やバックアップを置き換えるものではなく，それらが失敗・遅延した場合の復旧手段として多層防御を補完できる．今後は，記録パイプラインのスケーラビリティ改善と対応ファミリの拡充に取り組む．

参考文献

[1] Kaspersky: State of ransomware in 2025, https://securelist.com/state-of-ransomware-in-2025/116475/ (2025).

[2] Nugroho, Y.: Decrypting Encrypted files from Akira Ransomware (Linux/ESXI variant 2024) using a bunch of GPUs, https://tinyhack.com/2025/03/13/decrypting-encrypted-files-from-akira-ransomware-linux-esxi-variant-2024-using-a-bunch-of-gpus/ (2025).

[3] Kim, G., Kang, S., Baek, S., Kim, K. and Kim, J.: A Method for Decrypting Data Infected with Rhysida Ransomware, *arXiv preprint arXiv:2402.06440* (2024).

[4] Trend Micro: Ransomware Spotlight: Agenda, https://www.trendaisecurity.com/es-mx/resources-insights/deep-research/ransomware-spotlight-agenda (2026).

[5] SentinelOne: Hypervisor Ransomware | Multiple Threat Actor Groups Hop on Leaked Babuk Code to Build ESXi Lockers, https://www.sentinelone.com/labs/hypervisor-ransomware-multiple-threat-actor-groups-hop-on-leaked-babuk-code-to-build-esxi-lockers/ (2023).

[6] Halcyon: Open-Source Prince Ransomware Builder Used in Hospital Attack, https://www.halcyon.ai/blog/open-source-prince-ransomware-builder-used-in-mackay-memorial-hospital-attack (2025).

[7] Trend Micro: Ransomware Spotlight: DragonForce, https://www.trendaisecurity.com/en-us/resources-insights/deep-research/ransomware-spotlight-dragonforce (2025).

[8] Kolodenker, E., Koch, W., Stringhini, G. and Egele, M.: PayBreak: Defense Against Cryptographic Ransomware, *ACM AsiaCCS* (2017).

[9] Kim, H., Yoo, D., Kang, J.-S. and Yeom, Y.: Dynamic Ransomware Protection using Deterministic Random Bit Generator, *IEEE Conference on Application, Information and Network Security* (2017).

[10] Genç, Z. A., Lenzini, G. and Ryan, P. Y. A.: No Random, No Ransom: A Key to Stop Cryptographic Ransomware, *DIMVA* (2018).

[11] Bitam, S. and Alladoum, C.: Ransomware in the honeypot: how we capture keys with sticky canary files, Elastic Security Labs, https://www.elastic.co/security-labs/ransomware-in-the-honeypot-how-we-capture-keys (2024).

[12] Brodzik, A., Malec-Kruszyński, T., Niewolski, W., Tkaczyk, M., Bocianiak, K. and Loui, S.-Y.: Ransomware Detection Using Machine Learning in the Linux Kernel, *arXiv preprint arXiv:2409.06452* (2024).

[13] von der Assen, J., Feng, C., Huertas Celdrán, A., Oleš, R., Bovet, G. and Stiller, B.: GuardFS: a File System for Integrated Detection and Mitigation of Linux-based Ransomware, *arXiv preprint arXiv:2401.17917* (2024).

[14] Continella, A., Guagnelli, A., Zingaro, G., De Pasquale, G., Barenghi, A., Zanero, S. and Maggi, F.: ShieldFS: A Self-healing, Ransomware-aware Filesystem, *ACSAC* (2016).

[15] Lee, S., Park, M. and Kim, J.: Magniber v2 Ransomware Decryption: Exploiting the Vulnerability of a Self-Developed Pseudo Random Number Generator, *Electronics* (2021).

[16] Kim, G., Kim, S., Kang, S. and Kim, J.: A Method for Decrypting Data Infected with Hive Ransomware, *Journal of Information Security and Applications* (2022).

© 2026 Information Processing Society of Japan